

SeedHammer II — the pathological wallet

Backing up the constellation's **pathological example** from files: 11 keys, all four Bitcoin timelock kinds, and a hashlock. Host CLIs, the firmware GUI in the emulator, and the plates. No NFC anywhere.

Test material only — never put funds behind these keys.

The three masters are BIP-39's own published test vectors (abandon...about, legal winner...yellow, letter advice...above). They are public by construction.

The wallet

From mnemonic-toolkit's Examples §5, "Custom degrading-miniscript wallet — **the pathological example**". A four-tier vault that deliberately mixes every timelock kind Bitcoin has:

Tier	Spend condition	Timelock	Keys
1	3-of-3 + secret word	after(1000000) — absolute height	@0 @1 @2
2	2-of-3 + secret word	after(1893456000) — absolute time (2030-01-01)	@3 @4 @5
3	both	older(65535) — relative blocks (~455 d)	@6 @7
4	any 1 of 3	older(4255898) — relative time (~365 d)	@8 @9 @10

The secret word is opensesame; the descriptor commits to $H = \text{sha256}(\text{sha256}(\text{word}))$, shared by tiers 1 and 2. Reusing a *hash* across tiers is fine — it is not a key. The 11 keys come from three masters at divergent account indices (84' / 0' / 0' . . 3'), so no key is reused.

This is the case that actually forces chunking.

The 12-key multi(5,...) policy tried first encodes to 13 bytes and one md1 string — a keyless BIP-388 template does not pay per key. This one carries two 32-byte hash literals and four timelock arguments, and comes to **182 data symbols against a single-string cap of 80**. It is the first policy in this work that genuinely needs a chunk set.

Toolchain

```
$ /scratch/code/shibboleth/descriptor-mnemonic/target/release/md --version
md 0.13.0
[exit 0]

$ /scratch/code/shibboleth/mnemonic-key/target/release/mk --version
mk 0.12.1
[exit 0]

$ /scratch/code/shibboleth/mnemonic-secret/target/release/ms --version
ms 0.14.1
[exit 0]

$ /scratch/code/shibboleth/mnemonic-engrave/target/release/me --version
me 0.5.1
[exit 0]

$ /scratch/code/shibboleth/mnemonic-engrave/target/release/me-preview --version
me-preview 0.5.1
[exit 0]
```

The input files

THE POLICY

inputs/wallet-policy.txt

```
wsh(or_i(and_v(v:after(1000000),and_v(v:sha256(a84dce40975727c398023cfbd50d5db3b9662375521d0f1ac62dbd829b9a08ad),multi(3,@0/
<0;1>/*,@1/<0;1>/*,@2/<0;1>/*))),or_i(and_v(v:after(1893456000),and_v(v:sha256(a84dce40975727c398023cfbd50d5db3b9662375521d0
f1ac62dbd829b9a08ad),multi(2,@3/<0;1>/*,@4/<0;1>/*,@5/<0;1>/*))),or_i(and_v(v:older(65535),multi(2,@6/<0;1>/*,@7/<0;1>/*)),a
nd_v(v:older(4255898),multi(1,@8/<0;1>/*,@9/<0;1>/*,@10/<0;1>/*))))))
```

THE THREE MASTERS (SECRET — BIP-39 TEST VECTORS)

inputs/seeds/master-A.seed

```
abandon abandon abandon abandon abandon abandon aba
ndon abandon abandon abandon about
```

inputs/seeds/master-B.seed

```
legal winner thank year wave sausage worth useful legal win
ner thank yellow
```

inputs/seeds/master-C.seed

```
letter advice cage absurd amount doctor acoustic avoid lett
er advice cage above
```

THE ELEVEN KEYS

inputs/keys/key-00.xpub

```
# @0 – tier 1 (3-of-3, after height 1000000 + secret word)
# master A, origin [73c5da0a/84'/0'/0']
xpub6CatWdiZiodmUeTDp8LT5or8nmbKNcuYvz7WYksVFkKB4RHwCD3Xyuv
PEbvqAQY3rAPshWcMLoP2fMFMKHPJ4ZeZXYVUhLv1VMrjPC7PW6V
```

inputs/keys/key-01.xpub

```
# @1 – tier 1
# master A, origin [73c5da0a/84'/0'/1']
xpub6CatWdiZiodmYVtWLtEQsAg1H9ooS1bmsJUBwQ83FE1Fyk386FWcyic
JgEZv3quZSJKA5dh5Lo2PpubMGxCfZtRthV6ST2qquL9w3HSzcUn
```

inputs/keys/key-02.xpub

```
# @2 – tier 1
# master A, origin [73c5da0a/84'/0'/2']
xpub6CatWdiZiodmNGqcQxxjGN165QxTU4PwNNi9WrijYgYf7VxcmuFxoS
Rw3focZLgRDbjDjJbqZhPCTfkcaWmL9BuSw9YbKKJtcHgWery6
```

inputs/keys/key-03.xpub

```
# @3 – tier 2 (2-of-3, after time 1893456000 + secret word)
# master A, origin [73c5da0a/84'/0'/3']
xpub6CatWdiZiodmHurRokjbycCrxddTDJgTsyEaAQfKjkwBwUj79LAWG5
gHjMCQ87BeJc47MkubXuZdf45JZHk1qcr1GZ5EwREUDVdLVdPkEC
```

inputs/keys/key-04.xpub

```
# @4 – tier 2
# master B, origin [b8688df1/84'/0'/0']
xpub6DNfJehqF1LUUs9kwaqDu12Ajpz9psYVtbGhTykQo1CYdkkqV2vAyR4D
iWXSTTDujWHzVy1AtV6ENGKwgbLWqa4wXMR4ZmdpRjQBG5EgTV
```

inputs/keys/key-05.xpub

```
# @5 – tier 2
# master B, origin [b8688df1/84'/0'/1']
xpub6DNfJehqF1LUsoh1a4XLG3yuFY1oCRJd6Bkba97xvW6SwvY71Nc6Lvk
qfDLWuyCivJ4eMDxgFtiTxnazgp8rYVzQSKR3LEGRs9asBarqpe
```

inputs/keys/key-06.xpub

```
# @6 – tier 3 (both, older 65535 blocks)
# master B, origin [b8688df1/84'/0'/0']
xpub6DNfJehqF1LUwZ1DrFcQN365CDMPLSsrqpSwaz4eRrtYz9qWdsVH9Jq
sHpQ6yNGdG4VXGKbLgcXKkXtwTB5B4iCuHcmjomqR8z6NNxNpU51
```

inputs/keys/key-07.xpub

```
# @7 – tier 3
# master B, origin [b8688df1/84'/0'/3']
xpub6DNfJehqF1LUxb8gRSstbR9LcAxWgdV4V678z5FZ7BLftWzwQct3yZb
5eW4U8AVJwbKcZ2iegjjvKv2xggJpMBRkk3CE7bz7g4uMV7Qp3TU
```

inputs/keys/key-08.xpub

```
# @8 – tier 4 (any 1 of 3, older 4255898 = ~365d)
# master C, origin [28645006/84'/0'/0']
xpub6DBbzvudcQg2nS4tHskrm7FGkXL6arWxEYoZ1g1GWbaNdWRobcJmxH8
KQdjuzTZJtwDveibGd83eGzdGCR6NjSqP8p2Xx4wWQK7iGBycAm
```

inputs/keys/key-09.xpub

```
# @9 – tier 4
# master C, origin [28645006/84'/0'/1']
xpub6DBbzvudcQg2sPDRWpqfEn5VzPiwrd1zNes4aHmNUog9Jmc2fc2JSf
M2E39YzY2iakmpWRpa3rXhzGNmd4GKiJEZxmTcftQpwbGd9ihVks
```

inputs/keys/key-10.xpub

```
# @10 – tier 4
# master C, origin [28645006/84'/0'/2']
xpub6DBbzvudcQg2uk9BrxsuxCWjsYsbfPYkPahQfmTVABfJ4j8TRxUnRd5
eGgEXfgPJ63xcuTV9uny7pQBFBu3XKcN9rNxcNGRaDT9BD1gkBGZ
```

Host step 1 — it does not fit one string

```
$ cat /tmp/claude-1000/-scratch-code-shibboleth-mnemonic-engrave/22fd28a4-d68a-47d6-82b1-8a8570fb5417/scratchpad/journey2/inputs/wallet-policy.txt
wsh(or_i(and_v(v:after(1000000),and_v(v:sha256(a84dce40975727c398023cfbd50d5db3b9662375521d0f1ac62dbd829b9a08ad),multi(3,@0/<0;1>/*,@1/<0;1>/*,@2/<0;1>/*))),or_i(and_v(v:after(1893456000),and_v(v:sha256(a84dce40975727c398023cfbd50d5db3b9662375521d0f1ac62dbd829b9a08ad),multi(2,@3/<0;1>/*,@4/<0;1>/*,@5/<0;1>/*))),or_i(and_v(v:older(65535),multi(2,@6/<0;1>/*,@7/<0;1>/*))),and_v(v:older(4255898),multi(1,@8/<0;1>/*,@9/<0;1>/*,@10/<0;1>/*))))))
[exit 0]
```

```
$ /scratch/code/shibboleth/descriptor-mnemonic/target/release/md encode --group-size 0 wsh(or_i(and_v(v:after(1000000),and_v(v:sha256(a84dce40975727c398023cfbd50d5db3b9662375521d0f1ac62dbd829b9a08ad),multi(3,@0/<0;1>/*,@1/<0;1>/*,@2/<0;1>/*))),or_i(and_v(v:after(1893456000),and_v(v:sha256(a84dce40975727c398023cfbd50d5db3b9662375521d0f1ac62dbd829b9a08ad),multi(2,@3/<0;1>/*,@4/<0;1>/*,@5/<0;1>/*))),or_i(and_v(v:older(65535),multi(2,@6/<0;1>/*,@7/<0;1>/*))),and_v(v:older(4255898),multi(1,@8/<0;1>/*,@9/<0;1>/*,@10/<0;1>/*))))))
md: codec error: payload is 182 data symbols; the codex32 regular code caps single strings at 80 (use chunked encoding / --force-chunked)
[exit 1]
```

Host step 2 — so it is chunked, with an explicit origin

Encoded without `--path`, this policy's card carries no origin and `md` says so: its `wsh(or_i(...))` wrapper has no canonical default derivation path, so `md decode` only PARTIAL-decodes (exit 4, VERIFY-ME) and `me bundle` rejects the set outright. Supplying the origin the warning asks for is what makes the rest of the chain work.

```
$ /scratch/code/shibboleth/descriptor-mnemonic/target/release/md encode --group-size 0 --force-chunked --path bip84 wsh(or_i(and_v(v:after(1000000),and_v(v:sha256(a84dce40975727c398023cfbd50d5db3b9662375521d0f1ac62dbd829b9a08ad),multi(3,@0/<0;1>/*,@1/<0;1>/*,@2/<0;1>/*))),or_i(and_v(v:after(1893456000),and_v(v:sha256(a84dce40975727c398023cfbd50d5db3b9662375521d0f1ac62dbd829b9a08ad),multi(2,@3/<0;1>/*,@4/<0;1>/*,@5/<0;1>/*))),or_i(and_v(v:older(65535),multi(2,@6/<0;1>/*,@7/<0;1>/*))),and_v(v:older(4255898),multi(1,@8/<0;1>/*,@9/<0;1>/*,@10/<0;1>/*))))))
chunk-set-id: 0x02038
md1fqgpcqpz3m6jzzqqvzv5e4kqq0gfyqye4m2zdeeqfw4e8cwwqy08m65x4mvaevc3h25saqzxp5rc3jll8k8
md1fqgpcqpz3m6jzzqqvzv5e4kqq0gfyqye4m2zdeeqfw4e8cwwqy08m65x4mvaevc3h25saqzxp5rc3jll8k8
md1fqgpcqpz3m6jzzqqvzv5e4kqq0gfyqye4m2zdeeqfw4e8cwwqy08m65x4mvaevc3h25saqzxp5rc3jll8k8
md1fqgpcqpz3m6jzzqqvzv5e4kqq0gfyqye4m2zdeeqfw4e8cwwqy08m65x4mvaevc3h25saqzxp5rc3jll8k8
md1fqgpcqpz3m6jzzqqvzv5e4kqq0gfyqye4m2zdeeqfw4e8cwwqy08m65x4mvaevc3h25saqzxp5rc3jll8k8
note: stdout is a keyless descriptor template (no keys)
[exit 0]
```

What `--path` does and does not say.

It records ONE shared origin — `bip84' / 0' / 0'` produce a byte-identical chunk set here. These eleven keys actually sit at four account indices (`84' / 0' / 0' . . 3'`) across three masters, and each `mk1` card carries its own true `origin_path`. So the `md1`'s shared value is a default the key cards override; a restore that trusted the descriptor card alone would derive the wrong keys for `@3–@10`. Worth pinning with a restore test before this shape is recommended to anyone.

Host step 3 — the chunk set decodes back

Three cards, reassembled, give the same 11-key policy back — the round trip that makes the backup worth engraving.

```
$ /scratch/code/shibboleth/descriptor-mnemonic/target/release/md inspect md1fqgpcqpz3m6jzzqqvzv5e4kqq0gfyqye4m2zdeeqfw4e8cwwqy08m65x4mvaevc3h25saqzxp5rc3jll8k8 md1fqgpcqpz3m6jzzqqvzv5e4kqq0gfyqye4m2zdeeqfw4e8cwwqy08m65x4mvaevc3h25saqzxp5rc3jll8k8 md1fqgpcqpz3m6jzzqqvzv5e4kqq0gfyqye4m2zdeeqfw4e8cwwqy08m65x4mvaevc3h25saqzxp5rc3jll8k8
template: wsh(or_i(and_v(v:after(1000000),and_v(v:sha256(a84dce40975727c398023cfbd50d5db3b9662375521d0f1ac62dbd829b9a08ad),multi(3,@0/<0;1>/*,@1/<0;1>/*,@2/<0;1>/*))),or_i(and_v(v:after(1893456000),and_v(v:sha256(a84dce40975727c398023cfbd50d5db3b9662375521d0f1ac62dbd829b9a08ad),multi(2,@3/<0;1>/*,@4/<0;1>/*,@5/<0;1>/*))),or_i(and_v(v:older(65535),multi(2,@6/<0;1>/*,@7/<0;1>/*))),and_v(v:older(4255898),multi(1,@8/<0;1>/*,@9/<0;1>/*,@10/<0;1>/*))))))
n: 11
wallet-policy-mode: false
md1-encoding-id: 02038dc4fbb636e437178a2798f63fb9
wallet-descriptor-template-id: 5b48af35d4321a3ac18b43045e2523cc
wallet-policy-id: d3dda0f3a9ef2eef1f1de404b8a352a5
wallet-policy-id-fingerprint: 0xd3dda0f3
note: stdout is a keyless descriptor template (no keys)
[exit 0]
```

Obstacle 1 — mk cannot derive the policy stub from a chunked md1

Every mk1 key card must carry a 4-byte policy-id stub; `mk encode` refuses without one. The only automatic way to get it is `--from-md1`, and that path rejects a chunk:

```
$ /scratch/code/shibboleth/mnemonic-key/target/release/mk encode --xpub xpub6CatWdiZiodmUeTdp8LT5or8nmbKNcuYvz7WYksVFkKB4RHw
CD3XyuvPEbvqAQY3rAPshwCMLoP2fMFMKHPJ4ZeZXYVUhLv1VMrjPC7PW6V --origin-fingerprint 73c5da0a --origin-path m/84'/0'/0' --from-m
d1 md1fqgpcpqpz3m6jzzqqvzv5e4kqq0gfqye4m2zdeeqfw4e8cwwqy08m65x4mvaevc3h25saqzxp5rc3jll8k8 --group-size 0
error: md1 input rejected: wire-format version mismatch: got 9, expected 4
[exit 2]
```

`mk vendors md-codec 0.34.0`; the primary crate is at `0.42.0`. The "version 9" it will not parse is the chunked wire form.

WORKING AROUND IT — AND A SPEC/IMPLEMENTATION DIVERGENCE FOUND ON THE WAY

The stub is a property of the *descriptor*, not of the md1 string, so it is well defined for a chunked policy; `md inspect` prints the identities. But which one? `SPEC_mk_v0_1.md §3.3` is explicit — "the top 4 bytes of the MD-encoded policy's **WalletPolicyId**" — and today's `md` prints a field by exactly that name. **That is not the one mk uses.** Measured on a single-string wallet where `--from-md1` does work:

```
wallet-descriptor-template-id: 726a666305756435b7c52c5b3fc69c41
wallet-policy-id: f05e8a1c282f7740bbfd902a759b5577
policy_id_stubs (what mk embedded): 726a6663
```

The stub tracks the **template-id**, not the `wallet-policy-id`. Most likely a rename across versions — `mk vendors md-codec 0.34.0` and the primary is `0.42.0`, which now exposes both — but as it stands the spec sentence and the binary disagree about which identity a key card indexes.

```
$ bash -c /scratch/code/shibboleth/descriptor-mnemonic/target/release/md inspect md1fqgpcpqpz3m6jzzqqvzv5e4kqq0gfqye4m2zdeeq
fw4e8cwwqy08m65x4mvaevc3h25saqzxp5rc3jll8k8 md1fqgpcpqpz3m6jzzqqvzv5e4kqq0gfqye4m2zdeeqfw4e8cwwqy08m65x4mvaevc3h25saqzxp5rc3jll8k8
md1fqgpcpqpz3m6jzzqqvzv5e4kqq0gfqye4m2zdeeqfw4e8cwwqy08m65x4mvaevc3h25saqzxp5rc3jll8k8 md1fqgpcpqpz3m6jzzqqvzv5e4kqq0gfqye4m2zdeeqfw4e8cwwqy08m65x4mvaevc3h25saqzxp5rc3jll8k8
h5jk0d md1fqgpcpqpz3m6jzzqqvzv5e4kqq0gfqye4m2zdeeqfw4e8cwwqy08m65x4mvaevc3h25saqzxp5rc3jll8k8 md1fqgpcpqpz3m6jzzqqvzv5e4kqq0gfqye4m2zdeeqfw4e8cwwqy08m65x4mvaevc3h25saqzxp5rc3jll8k8
-id|template-id'
wallet-descriptor-template-id: 5b48af35d4321a3ac18b43045e2523cc
wallet-policy-id: d3dda0f3a9ef2eef1f1de404b8a352a5
wallet-policy-id-fingerprint: 0xd3dda0f3
[exit 0]
```

Following the binary rather than the sentence, this wallet's stub is **5b48af35**, supplied with `--policy-id-stub`.

Host step 4 — the eleven key cards

```
$ /scratch/code/shibboleth/mnemonic-key/target/release/mk encode --xpub xpub6CatWdiZiodmUeTDp8LT5or8nmbKNcuyvz7WyksVFkKB4RHwCD3XyuvPEbvqAQY3rAPshwCMLoP2fMFMKHPJ4ZeZXYVUhLv1VMrjPC7PW6V --origin-fingerprint 73c5da0a --origin-path m/84'/0'/0' --policy -id-stub 5b48af35 --group-size 0
mk1qpdw8zpqqsq4kj90x4eutks2qvzg3vs70mejkh622ws2kgdemj2cd8zwj2skzx2wq0qw70l4q99vdyh5x0z8v4yslsp8qs8lv4625hfhk862
mk1qpdw8zpp0f30mtxd65mvwcur9usdatwuqvq6z70r9nwrqk6xn6l8gy6nw970hj3pzy4nwdnyztz
note: stdout is watch-only — public keys only, cannot spend
[exit 0]

$ /scratch/code/shibboleth/mnemonic-key/target/release/mk decode mk1qpghz4pqqsq4kj90x4eutks2qvzg3vs70mejkh622ws2kgdemj2cd8zwj2skzx2wq0qw70l4q99vdyh5x0z8v4yslsp8qfcdga408tcwtxhq mk1qpghz4pp0f30mtxd65mvwcur9usdatwuqvq6z70r9nwrqk6xn6l8gy6nw970hj3pdka fng87nwvt
xpub:                xpub6CatWdiZiodmUeTDp8LT5or8nmbKNcuyvz7WyksVFkKB4RHwCD3XyuvPEbvqAQY3rAPshwCMLoP2fMFMKHPJ4ZeZXYVUhLv1VMrjPC7PW6V
origin_fingerprint:  73c5da0a
origin_path:         84'/0'/0'
policy_id_stubs:     5b48af35
chunks:              2 (long)
note: stdout is watch-only — public keys only, cannot spend
[exit 0]
```

Each key splits into **2 chunks**, so the eleven cards are 22 strings. Note the decode: the card carries the origin fingerprint and path, so the origins the descriptor card lacks are present in the bundle.

Host step 5 — the bundle validates, and names every plate

Every set verifies, the manifest emits, and the operator gets the number that matters before touching the machine.

```
$ /scratch/code/shibboleth/mnemonic-engrave/target/release/me bundle --in /tmp/claude-1000/-scratch-code-shibboleth-mnemonic-engrave/22fd28a4-d68a-47d6-82b1-8a8570fb5417/scratchpad/journey2/inputs/backup-strings.txt --preview /tmp/claude-1000/-scratch-code-shibboleth-mnemonic-engrave/22fd28a4-d68a-47d6-82b1-8a8570fb5417/scratchpad/journey2/out/plates --png --manifest /tmp/claude-1000/-scratch-code-shibboleth-mnemonic-engrave/22fd28a4-d68a-47d6-82b1-8a8570fb5417/scratchpad/journey2/out/manifest.json
me: rendered plate 1 ... (2-25 elided) ... → /tmp/claude-1000/-scratch-code-shibboleth-mnemonic-engrave/22fd28a4-d68a-47d6-82b1-8a8570fb5417/scratchpad/journey2/out/plates/plate-1.png
me: wrote manifest to /tmp/claude-1000/-scratch-code-shibboleth-mnemonic-engrave/22fd28a4-d68a-47d6-82b1-8a8570fb5417/scratchpad/journey2/out/manifest.json
me: backup needs 26 plates (25 public + ms1 on device):
  plate 1/26  md1 policy  → push via NFC & engrave
  plate 2/26  md1 policy  → push via NFC & engrave
  plate 3/26  md1 policy  → push via NFC & engrave
  plate 4/26  mk1 chunk 1/2 → push via NFC & engrave
  plate 5/26  mk1 chunk 2/2 → push via NFC & engrave
  plate 6/26  mk1 chunk 1/2 → push via NFC & engrave
  plate 7/26  mk1 chunk 2/2 → push via NFC & engrave
  plate 8/26  mk1 chunk 1/2 → push via NFC & engrave
  plate 9/26  mk1 chunk 2/2 → push via NFC & engrave
  plate 10/26 mk1 chunk 1/2 → push via NFC & engrave
  plate 11/26 mk1 chunk 2/2 → push via NFC & engrave
  plate 12/26 mk1 chunk 1/2 → push via NFC & engrave
  plate 13/26 mk1 chunk 2/2 → push via NFC & engrave
  plate 14/26 mk1 chunk 1/2 → push via NFC & engrave
  plate 15/26 mk1 chunk 2/2 → push via NFC & engrave
  plate 16/26 mk1 chunk 1/2 → push via NFC & engrave
  plate 17/26 mk1 chunk 2/2 → push via NFC & engrave
  plate 18/26 mk1 chunk 1/2 → push via NFC & engrave
  plate 19/26 mk1 chunk 2/2 → push via NFC & engrave
  plate 20/26 mk1 chunk 1/2 → push via NFC & engrave
  plate 21/26 mk1 chunk 2/2 → push via NFC & engrave
  plate 22/26 mk1 chunk 1/2 → push via NFC & engrave
  plate 23/26 mk1 chunk 2/2 → push via NFC & engrave
  plate 24/26 mk1 chunk 1/2 → push via NFC & engrave
  plate 25/26 mk1 chunk 2/2 → push via NFC & engrave
  plate 26/26 ms1 secret  → TYPE ON DEVICE (New > Input Seed > CODEX32); never via this tool
[exit 0]
```

Host step 6 — the seed, and the refusal that still holds

```
$ cat /tmp/claude-1000/-scratch-code-shibboleth-mnemonic-engrave/22fd28a4-d68a-47d6-82b1-8a8570fb5417/scratchpad/journey2/inputs/seeds/master-A.seed
abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon about
[exit 0]

$ /scratch/code/shibboleth/mnemonic-secret/target/release/ms encode --phrase abandon abandon abandon abandon abandon abandon
abandon abandon abandon abandon abandon about
msl0e ntrsq qqqqq qqqqq qqqqq qqqqq qqcj9 sxraq 34v7f
word count: 12
language: english (BIP-39 checksum valid)
passphrase: not stored in msl (record separately if used)
warning: stdout carries private key material (can spend) – redirect or encrypt (e.g. '> file.txt' or '| age -e ...')
[exit 0]

$ /scratch/code/shibboleth/mnemonic-engrave/target/release/me --in /dev/fd/63 --hex
me: refusing to emit msl over NFC: msl is secret seed entropy and must never be transmitted by radio (RF eavesdropping risk). Enter it by hand on the device: New > Input Seed > CODEX32.
[exit 3]
```


The 25 public plates

Three descriptor chunks, then eleven key cards at two chunks each.



plate 1 — md1 policy, chunk 1/3



plate 2 — md1 policy, chunk 2/3



plate 3 — md1 policy, chunk 3/3



plate 4 — @0 [73c5da0a/84/0/0'] chunk 1/2



plate 5 — @0 [73c5da0a/84/0/0'] chunk 2/2



plate 6 — @1 [73c5da0a/84/0/1] chunk 1/2



plate 7 — @1 [73c5da0a/84/0/1] chunk 2/2



plate 8 — @2 [73c5da0a/84/0/2] chunk 1/2



plate 9 — @2 [73c5da0a/84/0/2] chunk 2/2



plate 10 — @3 [73c5da0a/84/0/3] chunk 1/2



plate 11 — @3 [73c5da0a/84/0/3] chunk 2/2



plate 12 — @4 [b8688d1/84/0/0] chunk 1/2

The 25 public plates, continued



plate 13 — @4 [b8688df1/84/0/0] chunk 2/2



plate 14 — @5 [b8688df1/84/0/1] chunk 1/2



plate 15 — @5 [b8688df1/84/0/1] chunk 2/2



plate 16 — @6 [b8688df1/84/0/2] chunk 1/2



plate 17 — @6 [b8688df1/84/0/2] chunk 2/2



plate 18 — @7 [b8688df1/84/0/3] chunk 1/2



plate 19 — @7 [b8688df1/84/0/3] chunk 2/2



plate 20 — @8 [28645006/84/0/0] chunk 1/2



plate 21 — @8 [28645006/84/0/0] chunk 2/2



plate 22 — @9 [28645006/84/0/1] chunk 1/2



plate 23 — @9 [28645006/84/0/1] chunk 2/2



plate 24 — @10 [28645006/84/0/2] chunk 1/2



plate 25 — @10 [28645006/84/0/2] chunk 2/2

Plus three seed plates that are not here.

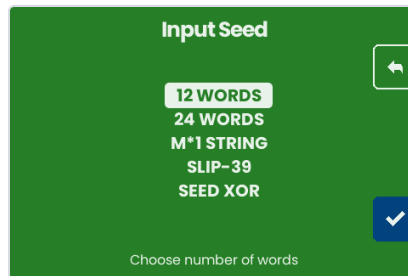
Each master is engraved from words typed on the machine. No host tool will render, transmit or preview them — so the real total is **28 plates**.

On the machine — the seed, typed

No NFC in this journey. The seed goes in the way the machine's own checklist says it must: by hand, on the device.



Boot: Backup Wallet



Input Seed — 12 words



Word 1 of 12



Three letters is enough: "1 match", auto-completed



All 12 accepted — the BIP-39 checksum holds



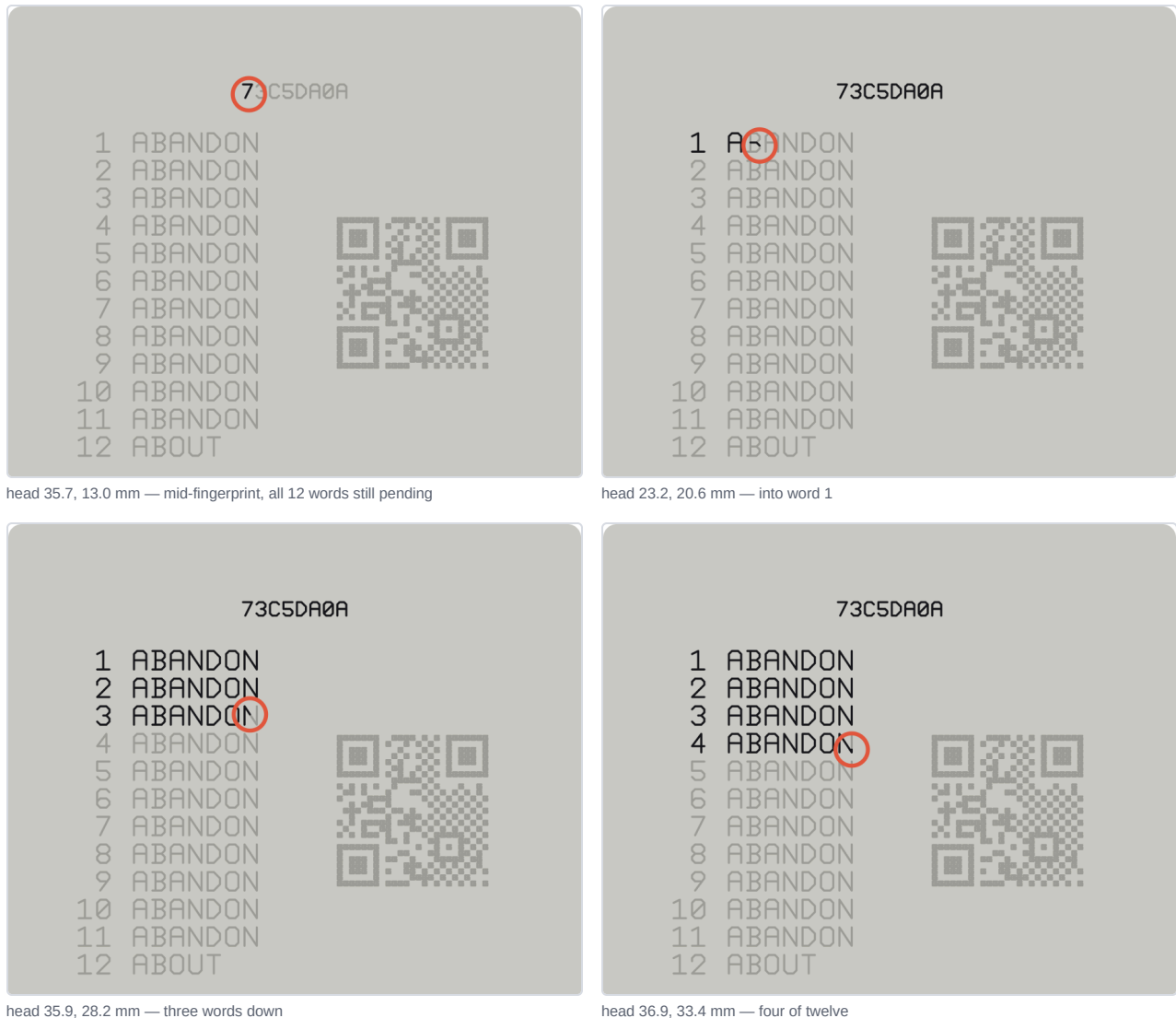
Optional BIP-39 passphrase, skipped

On the machine — the seed plate



The plate overlay, live

The whole planned layout is drawn in **grey** the moment the engrave begins, from the same PlanEngraving call the firmware makes. What the driver has actually stepped is filled in **black**, decoded from the real step stream. The red circle is the head. The title is the master fingerprint 73C5DA0A — the same one the descriptor's origins name.



What this run turned up

1. `mk encode --from-md1` cannot read a chunked md1.

wire-format version mismatch: got 9, expected 4, exit 2 — and a stub is mandatory, so for any policy large enough to need chunking the documented route to a key card does not work. `mk` vendors md-codec `0.34.0` against the primary's `0.42.0`; version 9 is the chunked wire form its copy predates. The provenance pin is what should have caught this. Workaround: read the identity out of `md inspect` and pass `--policy-id-stub` by hand — which nothing tells an operator to do, and which requires knowing the next finding.

2. The spec and the binary disagree about which identity the stub indexes.

`SPEC_mk_v0_1.md` §3.3 says the stub is the top 4 bytes of the **WalletPolicyId**, and `md` prints a field of exactly that name. Measured, `mk` embeds the top 4 bytes of the **wallet-descriptor-template-id** instead — `726a6663` where the `wallet-policy-id` began `f05e8a1c`. Probably a rename that landed in `md-codec` after 0.34.0, but the sentence and the code now name different things, and a stub is the index a recovering operator uses to tell wallets apart.

3. `--path` is required here, and it flattens.

Without it the descriptor card carries no origin, `md decode PARTIAL`-decodes (exit 4) and `me bundle` refuses the set. With it, everything validates — but it records a single shared origin while these eleven keys sit at four account indices. The true paths survive on the `mk1` cards; the descriptor card alone would restore `@3–@10` wrongly. This is the one finding here that is a *design* question rather than a defect: a restore test should pin which source wins.

4. `md encode` has no per-key origin flag.

`--key` takes a bare xpub and rejects an origin-annotated one (`base58check decode`), and `--fingerprint` carries no path. So a divergent-origin wallet cannot state its origins in the descriptor card at all — only the flattened form of finding 3 is expressible.

5. Carried over from the first journey:

presenting an NFC tag to a gathering flow freezes the emulator (filed as **F-126**). This journey avoids NFC entirely, so it is not exercised here.

Corrected from the first draft of this document.

It reported `me bundle`'s refusal as a tool defect and rendered the plates by calling the sidecar directly. That was wrong: the refusal was the consequence of omitting `--path`, which `md` had warned about at encode time. The plates and checklist here come from `me bundle --preview`.

Reproducing this document

```
cd <this directory>
bash transcript.sh > out/transcript.txt 2>&1 # every CLI block, refusals included
python3 build_pdf.py                        # this PDF
```

Emulator frames were captured by `POSTing canvas.toDataURL()` to a local receiver, so each screenshot is the device framebuffer exactly. Plate overlays are the page's own SVG, rendered with `rsvg-convert`.